

**UNIVERSIDAD PRIVADA DE TACNA**

**FACULTAD DE INGENIERÍA**

**ESCUELA DE INGENIERÍA DE SISTEMAS**



# **Guía Práctica de Laboratorio**

**“Registro de Contactos”**

**“PROGRAMACIÓN II”**

**DOCENTE:**

**Ing. Breno Luque Zuñiga**

**ESTUDIANTE:**

**Arturo Sebastian Cutipa Flores**

**TACNA – PERÚ**

**2026**



|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Guía de Laboratorio N° 02 Unidad 2 “Registro de Contactos ”</b> | <b>3</b>  |
| <b>I. INFORMACIÓN GENERAL</b>                                      | <b>3</b>  |
| Objetivos                                                          | 3         |
| Equipos, materiales, programas y recursos utilizados               | 3         |
| <b>II. MARCO TEÓRICO</b>                                           | <b>3</b>  |
| Windows Forms                                                      | 3         |
| Programación Orientada a Objetos                                   | 3         |
| CRUD                                                               | 3         |
| LINQ                                                               | 4         |
| DataGridView                                                       | 4         |
| <b>III. PROCEDIMIENTO</b>                                          | <b>4</b>  |
| Paso 1: Crear el proyecto                                          | 4         |
| Paso 2: Diseñar la interfaz gráfica                                | 4         |
| Paso 3: Creación de la clase Contacto                              | 5         |
| Paso 4: Declaración de la lista                                    | 5         |
| Paso 5: Programación del botón Agregar                             | 5         |
| Paso 6: Implementación de la visualización de datos                | 6         |
| Paso 7: Programación de la búsqueda                                | 6         |
| Paso 8: Programación de modificación                               | 7         |
| Paso 9: Programación de modificación                               | 7         |
| Paso 10: Ejecución y pruebas                                       | 8         |
| <b>IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS</b>                 | <b>9</b>  |
| Interfaz principal del sistema                                     | 9         |
| Registro de contactos                                              | 10        |
| Búsqueda de contactos                                              | 10        |
| Modificación de contactos                                          | 11        |
| Eliminación de contactos                                           | 11        |
| Interpretación General                                             | 12        |
| <b>V. CUESTIONARIO</b>                                             | <b>12</b> |
| 1. ¿Qué es un CRUD?                                                | 12        |
| 2. ¿Qué función cumple el DataGridView?                            | 12        |
| 3. ¿Qué es una clase en programación orientada a objetos?          | 12        |
| 4. ¿Qué es una lista genérica (List<T>)?                           | 13        |
| 5. ¿Qué ventaja ofrece LINQ?                                       | 13        |
| <b>CONCLUSIONES</b>                                                | <b>13</b> |
| <b>RECOMENDACIONES</b>                                             | <b>13</b> |
| <b>BIBLIOGRAFÍA</b>                                                | <b>13</b> |
| <b>WEBGRAFÍA</b>                                                   | <b>13</b> |



## **Guía de Laboratorio N° 02 Unidad 2 “Registro de Contactos”**

### **I. INFORMACIÓN GENERAL**

#### **Objetivos**

- Desarrollar una aplicación de escritorio utilizando Windows Forms.
- Aplicar programación orientada a objetos mediante una clase Contacto.
- Implementar operaciones CRUD (Crear, Leer, Modificar y Eliminar).
- Utilizar colecciones List<T> para almacenar datos temporalmente.
- Aplicar búsquedas utilizando LINQ.
- Mostrar información en un DataGridView.

#### **Equipos, materiales, programas y recursos utilizados**

- Computadora o laptop.
- Sistema operativo Windows.
- Visual Studio 2019/2022.
- Lenguaje de programación C#.
- Framework .NET Windows Forms.
- Editor de código integrado de Visual Studio.

### **II. MARCO TEÓRICO**

#### **Windows Forms**

Windows Forms es una tecnología de .NET que permite desarrollar aplicaciones de escritorio mediante interfaces gráficas utilizando controles visuales.

#### **Programación Orientada a Objetos**

La programación orientada a objetos permite representar entidades mediante clases y objetos.

En este proyecto se utilizó la clase Contacto.

#### **CRUD**

CRUD representa las operaciones básicas de manejo de datos:

- Create → Agregar
- Read → Mostrar
- Update → Modificar
- Delete → Eliminar



## LINQ

LINQ permite realizar consultas y búsquedas en colecciones de datos de forma sencilla y ordenada.

## DataGridView

Es un control utilizado para mostrar datos en forma de tabla dentro de una aplicación Windows Forms.

## III. PROCEDIMIENTO

### Paso 1: Crear el proyecto

Se inició Visual Studio y se creó un nuevo proyecto del tipo **Windows Forms App (.NET Framework)** con el nombre: Registro\_de\_Contactos

Posteriormente se creó el formulario principal donde se desarrolló toda la aplicación.

### Paso 2: Diseñar la interfaz gráfica

Desde el Toolbox se agregaron los controles necesarios para el registro y administración de contactos.

Se agregaron los siguientes componentes:

| Control  | Nombre       |
|----------|--------------|
| TextBox  | txtId        |
| TextBox  | txtNombre    |
| TextBox  | txtApellido  |
| TextBox  | txtTelefono  |
| TextBox  | txtDireccion |
| ComboBox | cboGenero    |
| TextBox  | txtBuscar    |
| Button   | btnAgregar   |
| Button   | btnModificar |
| Button   | btnEliminar  |

|              |               |
|--------------|---------------|
| Button       | btnBuscar     |
| DataGridView | dataGridView1 |

También se configuraron las propiedades de tamaño y posición para organizar correctamente la interfaz.

### Paso 3: Creación de la clase Contacto

Se agregó una nueva clase llamada Contacto.cs para representar la estructura de los datos de cada contacto.

```
CSharp
public class Contacto
{
    public int Id { get; set; }
    public string Nombre { get; set; }
    public string Apellido { get; set; }
    public string Telefono { get; set; }
    public string Direccion { get; set; }
    public string Genero { get; set; }
}
```

Esta clase permite aplicar programación orientada a objetos dentro del proyecto.

### Paso 4: Declaración de la lista

Dentro del formulario principal se declaró una lista genérica para almacenar temporalmente los contactos.

```
CSharp
List<Contacto> contactos = new List<Contacto>();
```

La lista permite guardar los registros ingresados por el usuario.

### Paso 5: Programación del botón Agregar

Se programó el botón Agregar para registrar nuevos contactos.

El sistema capturaba los datos ingresados en los TextBox y creaba un objeto de tipo Contacto.

```
CSharp
private void btnAgregar_Click(object sender, EventArgs e)
```

```
{
Contacto c = new Contacto();

c.Id = int.Parse(txtId.Text);
c.Nombre = txtNombre.Text;
c.Apellido = txtApellido.Text;
c.Telefono = txtTelefono.Text;
c.Direccion = txtDireccion.Text;
c.Genero = cboGenero.Text;

contactos.Add(c);

MostrarDatos();
}
```

Finalmente los datos eran mostrados en el DataGridView.

### **Paso 6: Implementación de la visualización de datos**

Se creó un método llamado MostrarDatos() encargado de actualizar automáticamente el DataGridView.

```
CSharp
private void MostrarDatos()
{
    dataGridView1.DataSource = null;
    dataGridView1.DataSource = contactos;
}
```

Esto permitió visualizar los registros almacenados en la lista.

### **Paso 7: Programación de la búsqueda**

Se implementó una búsqueda utilizando LINQ para encontrar contactos mediante el nombre ingresado.

```
CSharp
private void btnBuscar_Click(object sender, EventArgs e)
{
    var buscar = contactos.Where(x =>
        x.Nombre.ToLower().Contains(txtBuscar.Text.ToLower()))
        .ToList();

    dataGridView1.DataSource = null;
    dataGridView1.DataSource = buscar;
}
```

```
}
```

La búsqueda mostraba únicamente los contactos relacionados.

### **Paso 8: Programación de modificación**

Se programó el botón Modificar para actualizar la información de un contacto existente.

```
CSharp
private void btnModificar_Click(object sender, EventArgs e)
{
    var contacto = contactos.FirstOrDefault(x =>
        x.Nombre.ToLower() == txtBuscar.Text.ToLower());

    if (contacto != null)
    {
        contacto.Nombre = txtNombre.Text;
        contacto.Apellido = txtApellido.Text;
        contacto.Telefono = txtTelefono.Text;
        contacto.Direccion = txtDireccion.Text;
        contacto.Genero = cboGenero.Text;

        MostrarDatos();
    }
}
```

El sistema reemplaza los datos anteriores por la nueva información ingresada.

### **Paso 9: Programación de modificación**

Se implementó el botón Eliminar para borrar registros de la lista.

```
CSharp
private void btnEliminar_Click(object sender, EventArgs e)
{
    var contacto = contactos.FirstOrDefault(x =>
        x.Nombre.ToLower() == txtBuscar.Text.ToLower());

    if (contacto != null)
    {
        contactos.Remove(contacto);

        MostrarDatos();
    }
}
```

```
}
```

Después de eliminar el contacto, el DataGridView se actualizaba automáticamente.

### **Paso 10: Ejecución y pruebas**

Finalmente se ejecutó el sistema para verificar el correcto funcionamiento de todas las operaciones implementadas.

Se realizaron pruebas de:

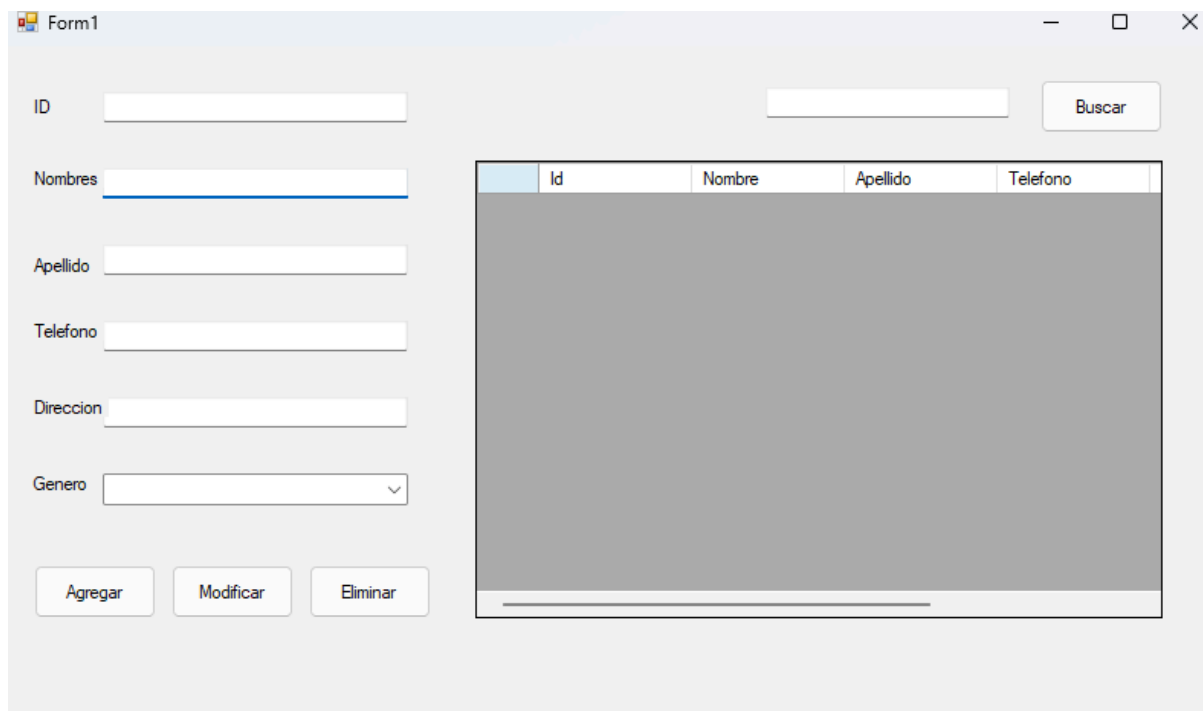
- Registro de contactos
- Búsqueda
- Modificación
- Eliminación
- Visualización de datos

Los resultados obtenidos fueron satisfactorios y el sistema funcionó correctamente.



## IV. ANÁLISIS E INTERPRETACIÓN DE RESULTADOS

### Interfaz principal del sistema:

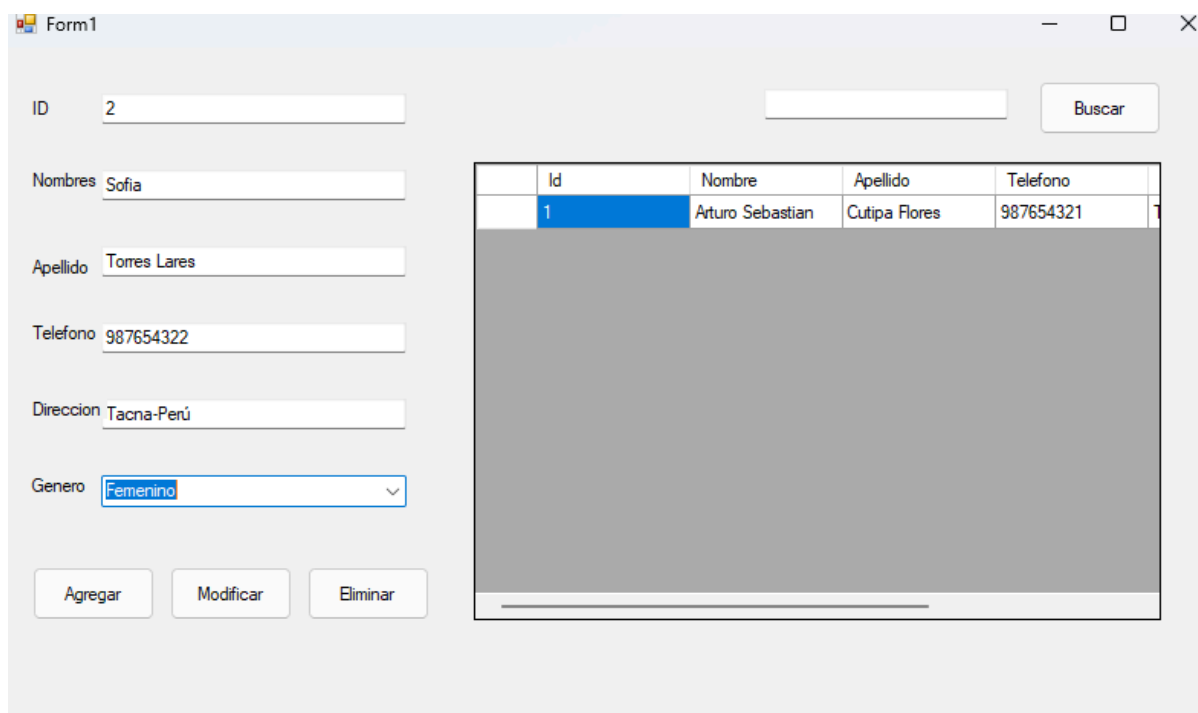


| Id | Nombre | Apellido | Telefono |
|----|--------|----------|----------|
|----|--------|----------|----------|

**Análisis:** En la imagen se observa la interfaz gráfica del sistema de registro de contactos desarrollado en Windows Forms.

La aplicación presenta controles organizados para el ingreso de datos, búsqueda y visualización de registros mediante un DataGridView.

La interfaz permite al usuario interactuar fácilmente con el sistema y realizar operaciones CRUD de manera sencilla.

**Registro de contactos:**

The screenshot shows a web application window titled 'Form1'. On the left, there are input fields for 'ID' (value: 2), 'Nombres' (value: Sofia), 'Apellido' (value: Torres Lares), 'Telefono' (value: 987654322), 'Direccion' (value: Tacna-Perú), and 'Genero' (value: Femenino). Below these fields are three buttons: 'Agregar', 'Modificar', and 'Eliminar'. On the right, there is a 'Buscar' button and a data grid. The data grid has columns: 'Id', 'Nombre', 'Apellido', and 'Telefono'. It contains two rows: Row 1 (Id: 1, Nombre: Arturo Sebastian, Apellido: Cutipa Flores, Telefono: 987654321) and Row 2 (Id: 2, Nombre: Sofia, Apellido: Torres Lares, Telefono: 987654322). The first row is highlighted in blue.

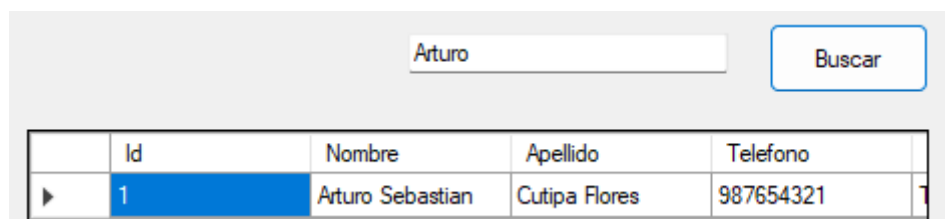
| Id | Nombre           | Apellido      | Telefono  |
|----|------------------|---------------|-----------|
| 1  | Arturo Sebastian | Cutipa Flores | 987654321 |
| 2  | Sofia            | Torres Lares  | 987654322 |

|  | Id | Nombre           | Apellido      | Telefono  |
|--|----|------------------|---------------|-----------|
|  | 1  | Arturo Sebastian | Cutipa Flores | 987654321 |
|  | 2  | Sofia            | Torres Lares  | 987654322 |

**Análisis:** Se verificó que el sistema registra correctamente la información ingresada por el usuario.

Los datos almacenados son mostrados automáticamente en el DataGridView, permitiendo visualizar los contactos registrados de forma ordenada.

Esto demuestra el correcto funcionamiento del botón Agregar y de la lista genérica utilizada para almacenar los datos.

**Búsqueda de contactos:**

The screenshot shows the search interface. At the top, there is a search input field containing the text 'Arturo' and a 'Buscar' button. Below this, the same data grid as in the previous screenshot is shown, but it only contains the first row (Id: 1, Nombre: Arturo Sebastian, Apellido: Cutipa Flores, Telefono: 987654321), which is highlighted in blue. This demonstrates the filtering results of the search.

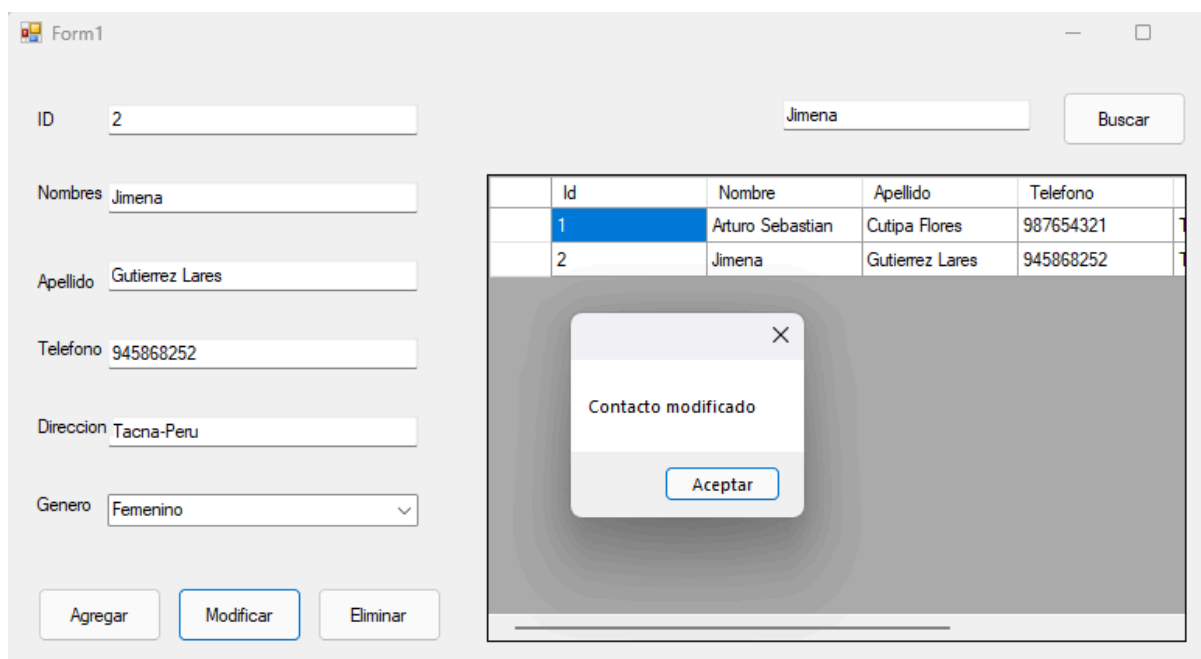
| Id | Nombre           | Apellido      | Telefono  |
|----|------------------|---------------|-----------|
| 1  | Arturo Sebastian | Cutipa Flores | 987654321 |

**Análisis:** El sistema realizó correctamente la búsqueda de contactos mediante el nombre ingresado por el usuario.

La implementación de LINQ permite filtrar los resultados y mostrar únicamente los registros relacionados con la búsqueda realizada.

Esto facilita la localización rápida de información dentro del sistema.

### Modificación de contactos:

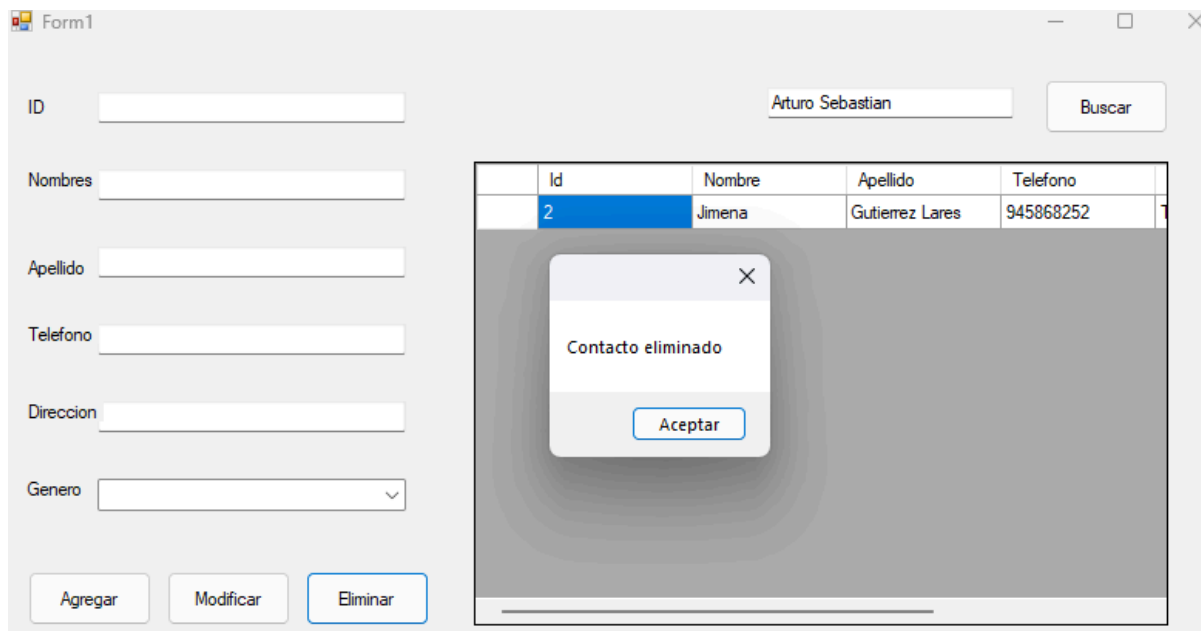


| Id | Nombre           | Apellido        | Telefono  |
|----|------------------|-----------------|-----------|
| 1  | Arturo Sebastian | Cutipa Flores   | 987654321 |
| 2  | Jimena           | Gutierrez Lares | 945868252 |

**Análisis:** Se comprobó que el sistema actualiza correctamente la información de un contacto existente.

Los cambios realizados fueron reflejados inmediatamente en el DataGridView, demostrando el correcto funcionamiento del proceso de modificación.

### Eliminación de contactos:



| Id | Nombre | Apellido        | Telefono  |
|----|--------|-----------------|-----------|
| 2  | Jimena | Gutierrez Lares | 945868252 |

**Análisis:** El sistema eliminó correctamente el contacto seleccionado y actualizó automáticamente el DataGridView.

Esto confirma que el proceso de eliminación funciona adecuadamente y mantiene actualizada la información mostrada en pantalla.

## **Interpretación General**

Durante las pruebas realizadas, el sistema respondió correctamente a todas las operaciones implementadas.

La aplicación permitió registrar, buscar, modificar y eliminar contactos sin presentar errores durante su funcionamiento final.

Además, se logró aplicar correctamente conceptos de:

- Programación orientada a objetos
- Listas genéricas (List<T>)
- LINQ
- Manejo de eventos
- Uso de DataGridView en Windows Forms

Los resultados obtenidos fueron satisfactorios y cumplieron con los objetivos planteados para la práctica de laboratorio.

## **V. CUESTIONARIO**

### **1. ¿Qué es un CRUD?**

CRUD es el conjunto de operaciones básicas utilizadas para manipular datos dentro de un sistema:

- Create → Crear
- Read → Leer
- Update → Modificar
- Delete → Eliminar

Estas operaciones permiten administrar información dentro de una aplicación.

### **2. ¿Qué función cumple el DataGridView?**

El DataGridView permite mostrar información en forma de tabla dentro de una aplicación Windows Forms.

Facilita la visualización y organización de los datos registrados por el usuario.

### **3. ¿Qué es una clase en programación orientada a objetos?**

Una clase es una plantilla que permite crear objetos con atributos y comportamientos específicos.

En este proyecto se utilizó la clase Contacto para representar la información de cada contacto registrado.

#### 4. ¿Qué es una lista genérica (List<T>)?

Es una colección dinámica que permite almacenar objetos del mismo tipo.

En el proyecto se utilizó para guardar temporalmente los contactos registrados.

#### 5. ¿Qué ventaja ofrece LINQ?

LINQ permite realizar consultas y búsquedas de manera sencilla y ordenada sobre colecciones de datos.

Facilita el filtrado y manipulación de información dentro de una aplicación.

### CONCLUSIONES

- Se desarrolló correctamente un sistema de registro de contactos utilizando Windows Forms y lenguaje C#.
- Se aplicaron conceptos de programación orientada a objetos mediante la creación de la clase Contacto.
- Las operaciones CRUD funcionaron correctamente durante las pruebas realizadas.
- El uso de LINQ facilitó la implementación de búsquedas dentro del sistema.
- El DataGridView permitió mostrar los datos de manera organizada y dinámica.
- El proyecto permitió reforzar conocimientos sobre eventos, listas genéricas y manipulación de datos en aplicaciones de escritorio.

### RECOMENDACIONES

- Validar los datos ingresados por el usuario para evitar errores.
- Implementar almacenamiento en base de datos para conservar la información permanentemente.
- Mejorar el diseño visual de la interfaz gráfica.
- Agregar mensajes de confirmación antes de eliminar registros.
- Incorporar más filtros de búsqueda para optimizar el sistema.

### BIBLIOGRAFÍA

- Deitel, P. C#. Cómo programar. Pearson Educación.
- Microsoft Corporation. Manual de programación en C#.
- Joyanes Aguilar, L. Fundamentos de Programación.

### WEBGRAFÍA

- Microsoft Learn. Documentación oficial de C#.
- Microsoft Learn. Guía de LINQ en .NET.
- Microsoft Learn. Windows Forms Documentation.